Table of Contents

Introduction **2**

npm tools **3**

npm install: install node modules in package.json *3*

npm run build: Building for Production & Error Checking *3*

Running and Debugging the Development Server **5**

Use the Browser built-in Developer Tools **7**

Use the browser extension React Developer Tools **9**

Demo: React Debugging

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

Introduction

npm Tools

- **npm install**: Installs node modules listed in package.json. Reinstalls missing modules if needed.
- **npm run build**: Builds the project and generates the build folder. Displays warnings and errors (e.g., unused imports).
- **npm start**: Builds and runs the frontend project, displaying warnings and errors in the terminal and browser.

Browser Built-in Developer Tools

- **Elements**: Inspect components and styles on the web page.
- **Console**: View logs and outputs from the code.
- **Sources**: Set breakpoints and inspect variables. Use buttons and scopes to control debugging.

React Developer Tools Extension

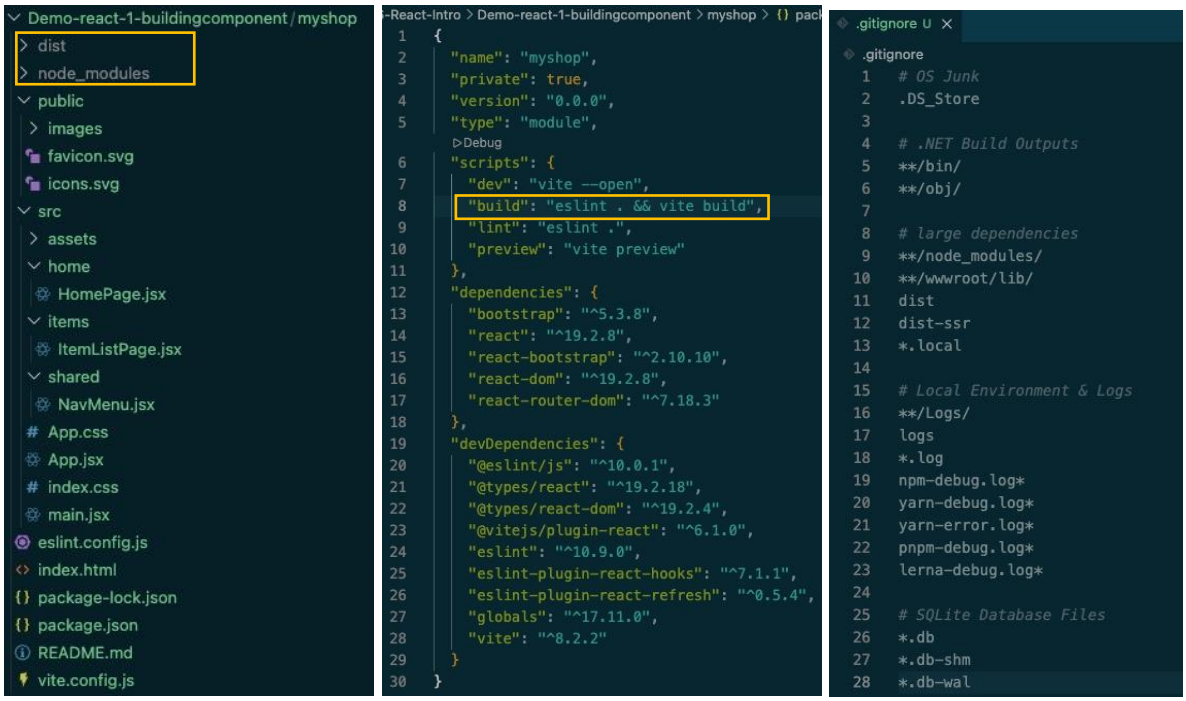
- **Components**: View component tree, inspect and modify props and states, and use the element picker.
- **Profiler**: Measure and analyse rendering performance of React components.

npm tools

The npm, node package manager, offers many tools.

npm install: install node modules in package.json

The node_modules can be store all the installed modules documented in package.json. If some modules are missing, or the complete folder is missing. Run `npm install` to reinstall it. The Vite react build tool will generate a .gitignore automatically. You can edit it to suit for our purpose. Or we can also use our previous .gitignore and add the `dist` folder.



npm run build: Building for Production & Error Checking

When developing a React application, `npm run dev` serves your code rapidly without enforcing strict code checks. To verify that your application is free of errors, unused variables, or broken React hook usage, you should build your project using `npm run build`.

1. Configuring package.json to Enforce Linting

By default, Vite's build process only bundles files and will skip code style checks. To ensure `npm run build` inspects your code for potential errors and warnings before generating build artifacts, update the `build` script in your package.json to include ESLint:

Open package.json and modify the "build" script under "scripts" (line8 in the above middle figure)

2. Running the Build Command

To compile the project, run:

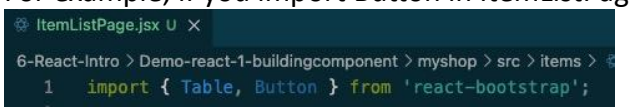
```
npm run build
```

When successful, Vite compiles and optimises your code into the **dist** folder. The dist (distribution) folder contains the final intermediate build files ready to be deployed to a production web server.

3. Catching Build Warnings and Errors

Because `eslint .` now runs prior to vite build, the process will halt immediately if unused variables, bad imports, or syntax violations are found.

For example, if you import Button in ItemListPage.jsx but never use it:



Running `npm run build` will catch the unused import and terminate with an error output:

```
> myshop@0.0.0 build
> eslint . && vite build

/Users/username/myshop/src/items/ItemListPage.jsx
  1:17  error  'Button' is defined but never used  no-unused-vars

✖ 1 problem (1 error, 0 warnings)
```

Tip: You must fix all ESLint errors (e.g., removing unused imports or variables) before the build process will generate the dist folder successfully.

Running and Debugging the Development Server

Starting the Application (npm run dev)

In Vite-based React projects, you start your development server using:

```
npm run dev
```

In day-to-day development, **you do not need to run npm run build repeatedly**. The command **npm run dev** starts Vite's instant development server with Hot Module Replacement (HMR). It automatically tracks changes, displays errors and warnings in real time, and updates your browser immediately without requiring a manual rebuild or server restart.

Real-Time Error Detection

Vite and React provide three layers of feedback when you make a mistake (for example, returning multiple adjacent JSX elements like putting an <h1> outside the main without a React Fragment wrapper in ItemListPage.jsx):

1. **In VS Code:** Your editor highlights syntax violations in real time with red squiggly lines under the invalid code.



```
23    return (
24      <h1>Items</h1>
25      <div>
26        <Table striped bordered hover>
27        <thead>
28          <tr>
29            <th>ID</th>
30            <th>Name</th>
31            <th>Price</th>
32            <th>Descriptions</th>
33            <th>Images</th>
34          </tr>
35        </thead>
36        <tbody>
37          {items.map(item => (
38            <tr key={item.ItemId}>
39              <td>{item.ItemId}</td>
40              <td>{item.Name}</td>
41              <td>{item.Price} NOK</td>
42              <td>{item.Description}</td>
43              <td><img src={item.ImageUrl} alt={item.Name} width="120" /></td>
44            </tr>
45          ))}
46        </tbody>
47        </Table>
48      </div>
49    );
50  };
```

2. **In the Terminal:** The running npm run dev terminal logs the precise file, line number, and error message immediately:

```
[USER]:myshop % npm run dev

> myshop@0.0.0 dev
> vite --open

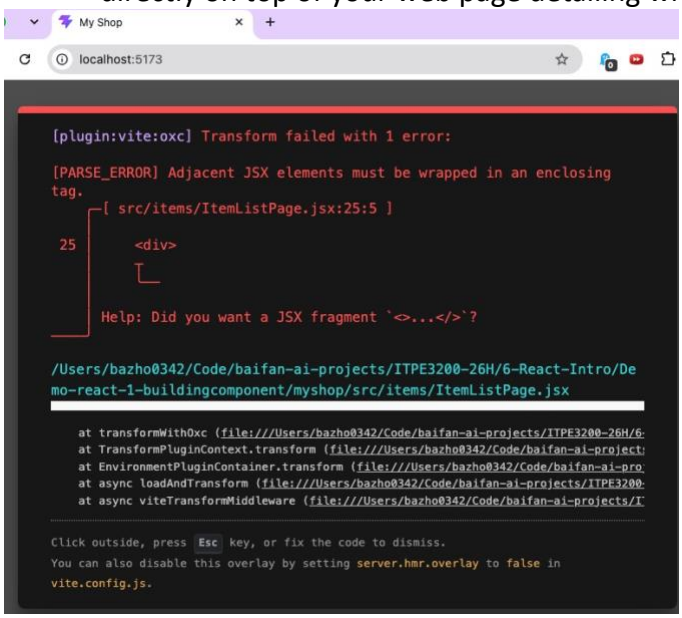

  VITE v8.2.2  ready in 92 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
6:58:34 PM [vite] (client) Pre-transform error: Transform failed with 1 error:

[PARSE_ERROR] Adjacent JSX elements must be wrapped in an enclosing tag.
    ╭─[ src/items/ItemListPage.jsx:25:5 ]
    │
 25 │       <div>
    │       ┬─
    │       ╰─
    │
    │     Help: Did you want a JSX fragment `<>...</>`?
────╯

  Plugin: vite:oxc
  File: /Users/bazho0342/Code/baifan-ai-projects/ITPE3200-26H/6-React-Intro/Demo-react-1-
buildingcomponent/myshop/src/items/ItemListPage.jsx
6:58:34 PM [vite] Internal server error: Transform failed with 1 error:
```

3. **In the Browser Overlay:** Instead of crashing silently, Vite displays a red error overlay directly on top of your web page detailing what failed and where.



Fixing Errors with Hot Reloading

To fix the issue:

- Return to your code in VS Code and fix the syntax (e.g., wrap the elements inside a parent or <> ... </>).
- Save the file (Ctrl + S or Cmd + S).

The terminal will clear the error, the browser overlay will disappear automatically, and your page will update instantly (if not, refresh it). **There is no need to stop the development server (Ctrl + C) or restart npm run dev.**

.

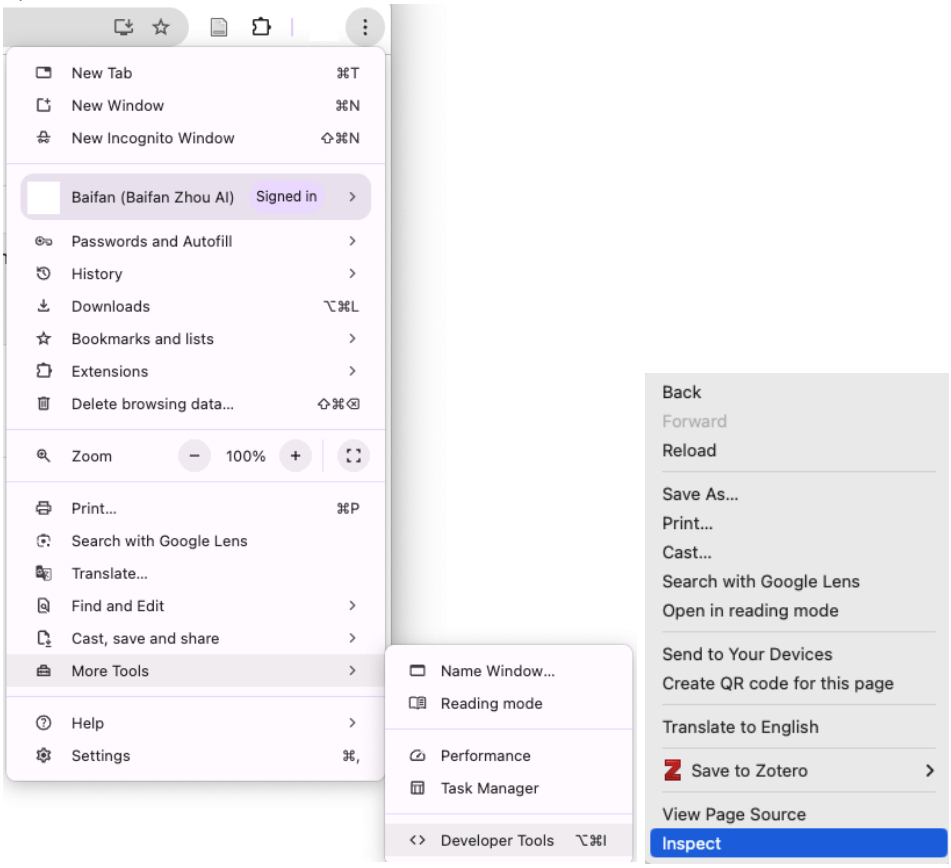
Use the Browser built-in Developer Tools

There are multiple ways to open the browser built-in developer tools. We take Chrome as example. Other browsers are similar. Use Google to find out.

1, click the three dots → more tools → Developer Tools

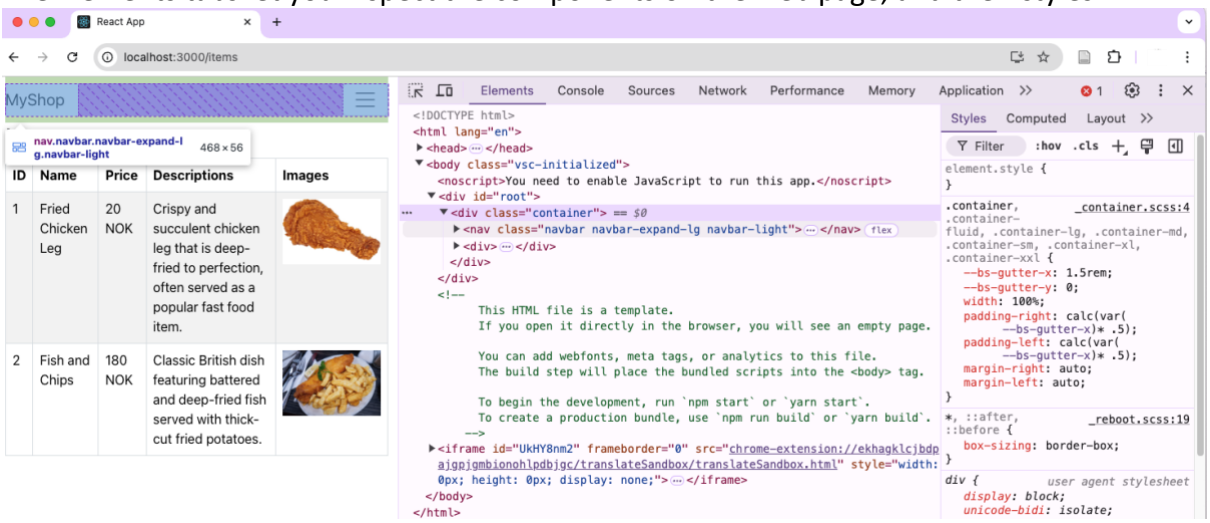
2, right-click any webpage in an empty place → Inspect

3, Press F12.



Most commonly used are the Elements, Console, and Sources tabs.

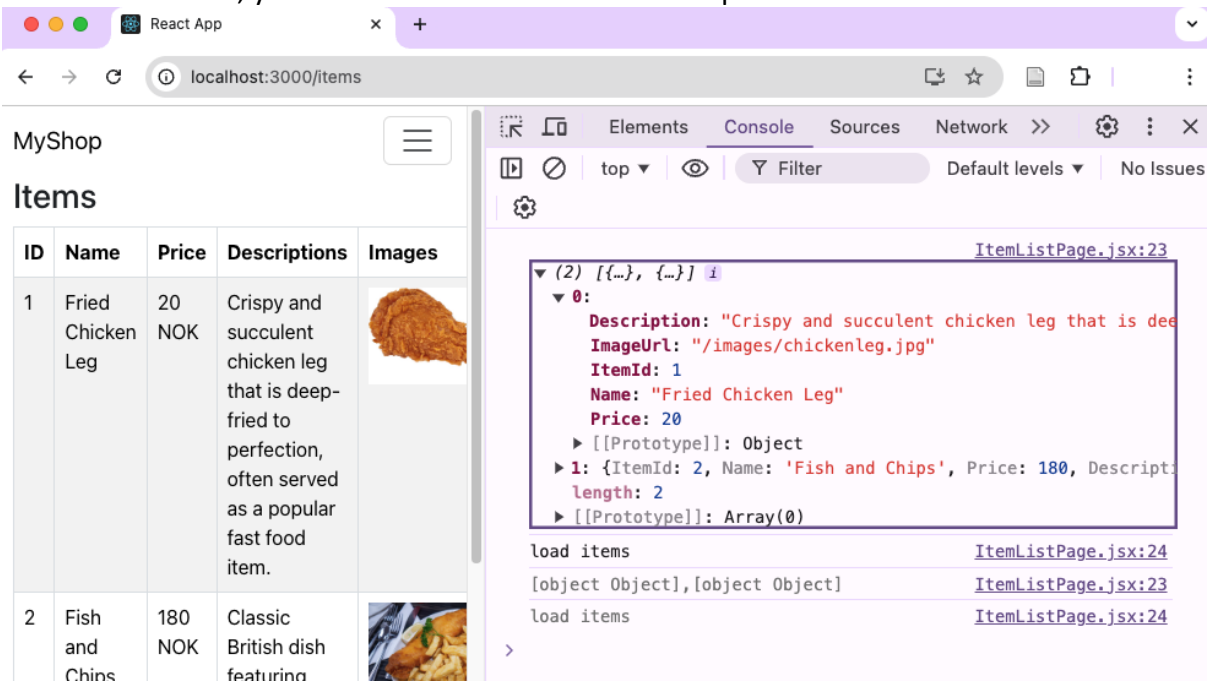
The Elements tabs let you inspect the components on the web page, and their styles.



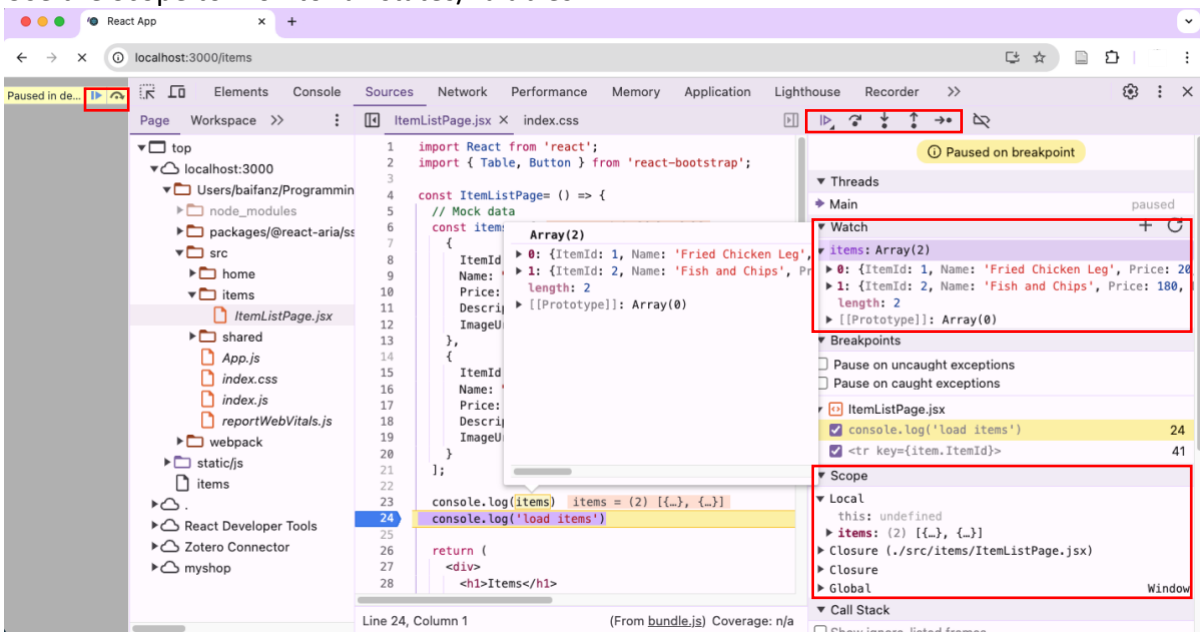
The Console tab outputs the logs during the running.
For example, I add these lines in ItemListPage.jsx.

```
23    console.log(items)
24    console.log('load items')
```

In the Console tab, you will be able to see these are output in the console.



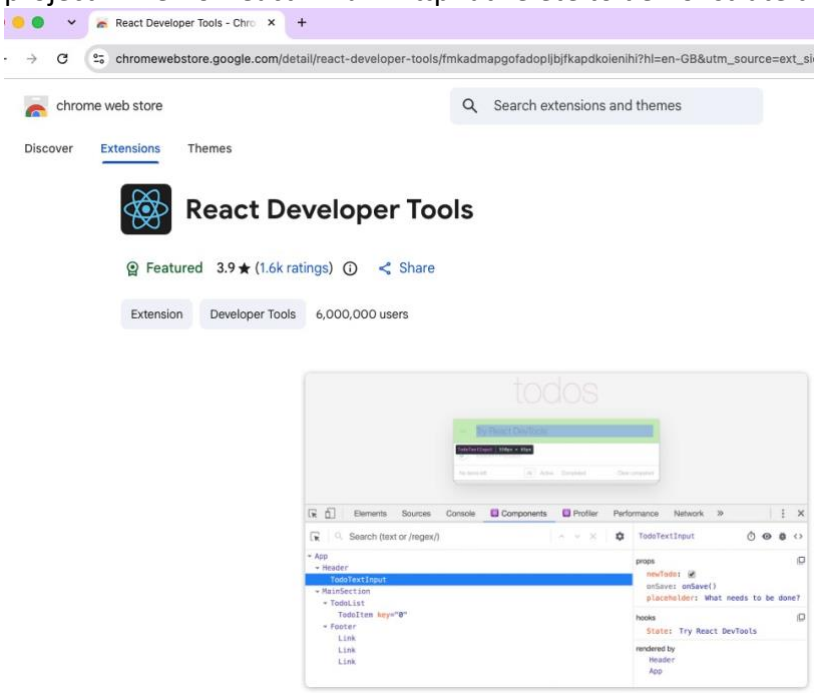
The Sources tab allows you to set breakpoints in the frontend code. In the following example, we add a breakpoint at line 24. If the ItemListPage is loaded, it stops at Line24.
Then we can hover over variables to see their content.
Use the buttons in red rectangles to control the debugging flow.
Use the Watch to add variables to observe.
Use the Scope to monitor all states/vaiables.



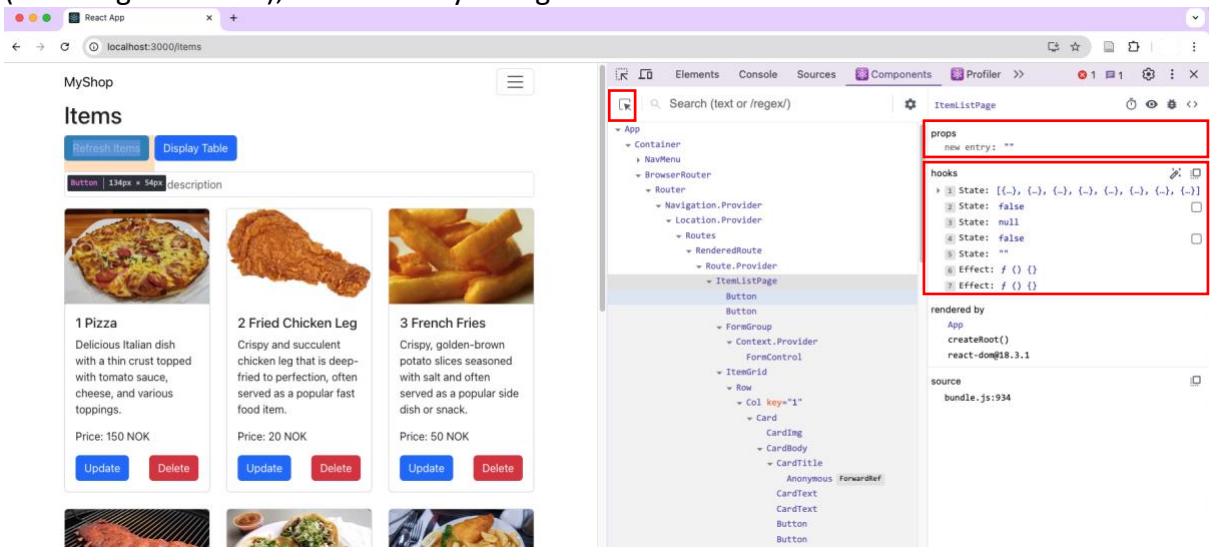
There are many good uses of the Developer Tools. You can explore a bit by yourself. ☺

Use the browser extension React Developer Tools

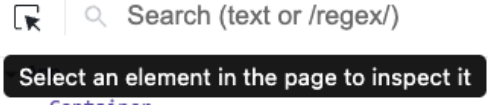
Instal the browser extension: React Developer Tools. It should be available for many popular Browsers such as Chrome, Firefox, Edge, Safari (installation is different).
It's easier to see what the tool is capable if you have more complicated project. I use the project in Demo-React-7-AJAXHttpPutDelete to demonstrate the tool.



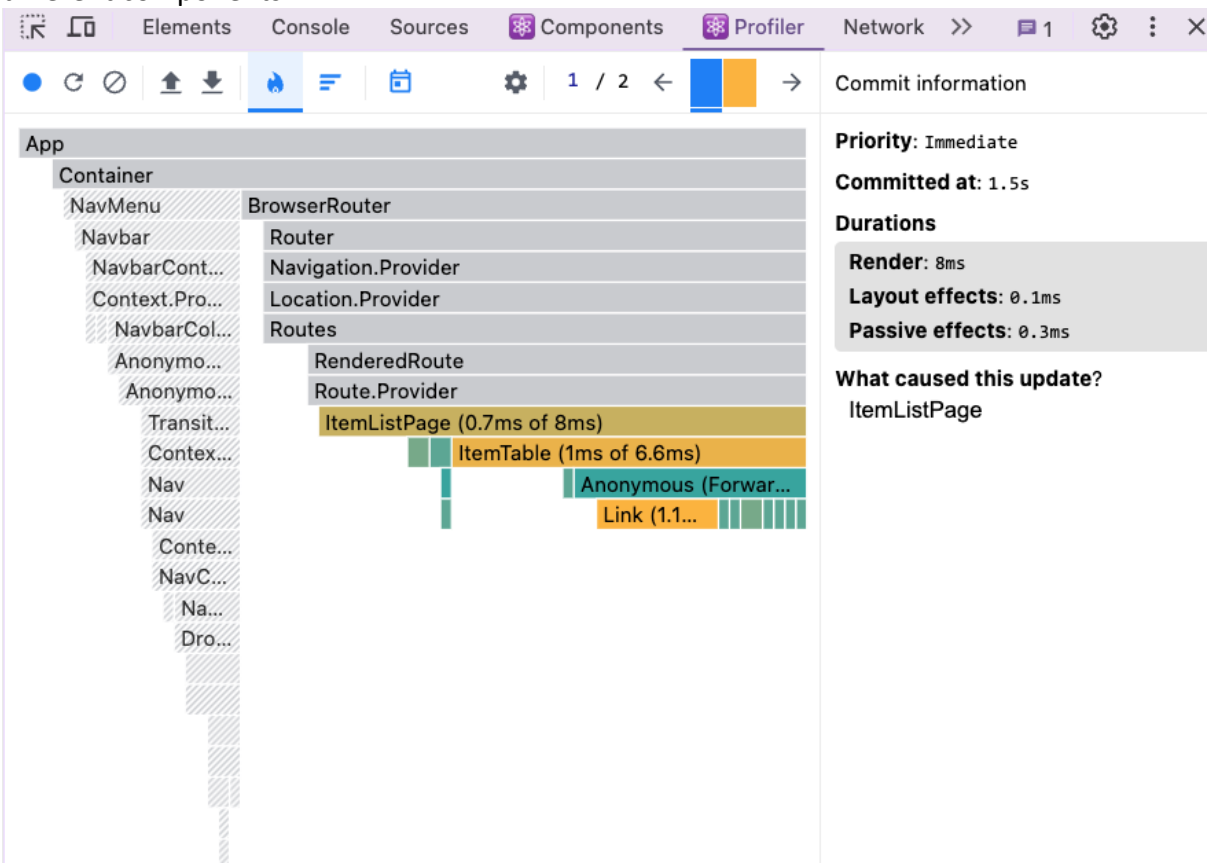
Two new tabs appear: Components, and Profiler.
In the Components tab, you can a tree of components. Click any of them, you can see the component highlighted on the web page, and view the input properties, all the hooks (including the states), and manually change the Boolean states see the effect.



There is also an element picker in the red rectangle that allows you to pick up element, and a search bar for you to find elements.



The Profiler tab allows you to measure rendering performance of a React tree programmatically. You record a period of page running, and analyse the response time of different components.



Read more: https://react.dev/learn/react-developer-tools

Congratulations! You have learnt a handful of tools for debugging the React frontend project! ☺